Design Patterns in the Fundraising Platform

Complete pattern guide - .NET APIs - microservices - payments - messaging - resilience

Pattern Study Map

- Creational patterns
- Structural patterns
- Behavioral patterns
- Application / enterprise patterns
- CQRS / commands / mediator
- Domain events / Outbox / Inbox
- Idempotency & resilience
- Saga / distributed workflows
- Caching & API Gateway
- External provider isolation
- Patterns by microservice
- Anti-pattern / overuse guidance
- Recommended pattern set
- Final architecture memory map

Why We Are Using Design Patterns

A design pattern is not simply a class structure that we add to a project so that we can say:

> "This project uses 20 design patterns."

A design pattern is a reusable solution to a recurring software-design problem.

For this project, every pattern should answer:

EXAMPLE / FLOW

```
What problem do we have?

Why does this pattern help?

Where exactly will we use it?

Which microservice owns it?

What classes/interfaces will demonstrate it?

What should we NOT do with it?
```

Our architecture is:

EXAMPLE / FLOW

```
                    Angular
                       |
                       v
                API Gateway later
                       |
        +--------------+--------------+
        |              |              |
        v              v              v
Fundraising API   Payment API   Reporting API
        |              |              |
        v              v              v
FundraisingDb      PaymentDb     ReportingDb
        |
        v
Outbox / Events
        |
        v
 Azure Service Bus
```

The design patterns should support this architecture rather than make it unnecessarily complicated.

Pattern Categories

Five Pattern Categories

Creational	Structural	Behavioral	Application	Distributed
How objects are created	How objects are composed	How behavior varies	How use cases are organized	How services coordinate

We will cover patterns in five groups:

EXAMPLE / FLOW

```
Creational
Structural
Behavioral
Application / Enterprise
Distributed-System / Microservice
```

The reusable patterns discussed here fit mainly in the first four groups.

Our fundraising platform will also add several distributed-system patterns because we have:

EXAMPLE / FLOW

```
Fundraising Service

Payment Service

Reporting Service

Service Bus

external payment provider
```

Complete Pattern Map

Pattern	Use in Our Project	Importance
Factory	Payment providers, exception mapping, DbContext factory	Core
Singleton	Immutable/thread-safe shared services	Core where appropriate
Builder	Application startup, test builders	Core
Repository	Aggregate persistence	Core
Unit of Work	Database transaction boundary	Core
Decorator	Logging/metrics/payment gateway	Core
Adapter	External payment providers	Core
Facade	Application/business service boundary	Useful
Proxy	Specialized scenarios	Optional
Strategy	Payment/fraud/exception behavior	Core
Template Method	Background jobs/import processing	Useful
Chain of Responsibility	Middleware/request processing	Core
Observer	Domain events/events	Core
Dependency Injection	Entire architecture	Core
Service Layer	Application/business orchestration	Core
DTO	API contracts	Core
Mapper	DTO/domain mapping	Core
Options	Configuration	Core
MVC	Controller-based Web API	Core concept
Policy Scheme	Authentication selection	Advanced
Exception Handler	Centralized error handling	Core
CQRS	Commands and queries	Core
Mediator	Request/handler decoupling	Useful
Specification	Reusable query/business rules	Optional
Result	Expected business failures	Core
Domain Event	Internal business event communication	Core
Outbox	Reliable external event publishing	Essential
Inbox / Idempotent Consumer	Duplicate event protection	Essential
Idempotency	Payment/API retry protection	Essential

Pattern	Use in Our Project	Importance
Saga / Process Manager	Cross-service workflows	Advanced/Core later
Retry	Transient failures	Core
Circuit Breaker	Dependency failures	Core
Timeout	External-service protection	Core
Bulkhead	Resource isolation	Advanced
Cache-Aside	Redis/read caching	Useful
API Gateway	Microservice entry point	Core later
Anti-Corruption Layer	External payment provider isolation	Very useful
State Pattern	Payment lifecycle	Optional advanced
Command Pattern	CQRS commands	Core through CQRS
Builder for Tests	Domain/test object creation	Very useful

CREATIONAL PATTERNS

1. Factory Pattern

Simple Definition

The Factory pattern creates an object without forcing the caller to know exactly how that object is constructed.

Instead of:

EXAMPLE / FLOW

```
Controller
   |
   v
new ProviderAPaymentGateway(...)
```

the caller asks:

EXAMPLE / FLOW

```
PaymentGatewayFactory
```

to give it the correct implementation.

Best Business Use — Payment Provider Factory

Suppose ABC Children's Foundation uses:

EXAMPLE / FLOW

```
Provider A
```

while another tenant uses:

EXAMPLE / FLOW

```
Provider B.
```

We do not want:

EXAMPLE / FLOW

```
if (provider == "A")
{
    // 100 lines
}
else if (provider == "B")
{
    // another 100 lines
}
```

inside `PaymentsController`.

Instead:

EXAMPLE / FLOW

```
                IPaymentGateway
                      ^
                      |
          +-----------+-----------+
          |                       |
ProviderAPaymentGateway    ProviderBPaymentGateway
          ^                       ^
          |                       |
          +-----------+-----------+
                      |
              PaymentGatewayFactory
```

Contract:

EXAMPLE / FLOW

```
public interface IPaymentGateway
{
    string ProviderName { get; }

    Task<PaymentGatewayResult> CreatePaymentAsync(
        PaymentGatewayRequest request,
        CancellationToken cancellationToken);
}
```

Factory:

EXAMPLE / FLOW

```
public interface IPaymentGatewayFactory
{
    IPaymentGateway GetGateway(string providerName);
}
```

Business flow:

EXAMPLE / FLOW

```
Payment request
      |
      v
MerchantAccount
Provider = A
      |
      v
PaymentGatewayFactory
      |
      v
ProviderAPaymentGateway
      |
      v
External provider
```

This is one of the strongest Factory Pattern examples in the project.

• ExceptionResultFactory

A practical exception-mapping example:

EXAMPLE / FLOW

```
ExceptionResultFactory
```

can also fit.

For example:

EXAMPLE / FLOW

```
EntityNotFoundException
        |
        v
ExceptionResultFactory
        |
        v
404 ProblemDetails
```

or:

EXAMPLE / FLOW

```
ConcurrencyConflictException
        |
        v
ExceptionResultFactory
        |
        v
409 ProblemDetails
```

However, in modern ASP.NET Core I would combine this concept with:

EXAMPLE / FLOW

```
IExceptionHandler
+
ProblemDetails
```

rather than building a giant custom exception framework.

• DbContextFactory

`IDbContextFactory<TContext>` has a real purpose.

Normal API request:

EXAMPLE / FLOW

```
Scoped DbContext
```

is sufficient.

But a background worker such as:

EXAMPLE / FLOW

```
OutboxPublisher
```

may need to create short-lived DbContexts repeatedly.

Example:

EXAMPLE / FLOW

```
Outbox BackgroundService
       |
       v
IDbContextFactory<FundraisingDbContext>
       |
       v
create context
       |
       v
process batch
       |
       v
dispose
```

This is a legitimate Factory use.

When NOT to Use Factory

Do not create:

EXAMPLE / FLOW

```
GiftFactory
CampaignFactory
FundFactory
TaskFactory
```

when construction is simply:

EXAMPLE / FLOW

```
new Task(...)
```

and there is no meaningful construction complexity.

Factories should solve construction/selection complexity.

2. Singleton Pattern

Definition

Singleton means one object instance exists for the lifetime of the application process.

In ASP.NET Core this is normally achieved through DI:

EXAMPLE / FLOW

```
services.AddSingleton<IClock, SystemClock>();
```

Good Business Uses

Possible project examples:

EXAMPLE / FLOW

```
immutable payment-provider metadata

OpenAPI configuration options

shared TimeProvider

thread-safe serialization configuration

immutable provider registry
```

FrozenDictionary Example

A similar immutable-configuration approach uses:

EXAMPLE / FLOW

```
FrozenDictionary
```

as singleton-like immutable state.

We can use a similar concept for immutable provider metadata:

EXAMPLE / FLOW

```
Provider Code
→ Configuration Metadata
```

Example:

EXAMPLE / FLOW

```
VISA
→ Card Provider

PAYPAL
→ Wallet Provider
```

A frozen/immutable dictionary is a strong fit because:

EXAMPLE / FLOW

```
created once

read many times

never modified

thread-safe for reads
```

• Important Singleton Rule

Never put:

EXAMPLE / FLOW

```
CurrentTenant

CurrentUser

DbContext

current payment

request DTO
```

in a singleton.

Those values change per request.

They belong in:

EXAMPLE / FLOW

```
Scoped services.
```

• 3. Builder Pattern

Your existing projects use:

EXAMPLE / FLOW

```
HostBuilder
WebApplicationBuilder
```

which are Builder-pattern examples.

Our application startup uses:

EXAMPLE / FLOW

```
var builder =
    WebApplication.CreateBuilder(args);
```

Then progressively:

EXAMPLE / FLOW
```
builder.Services.AddAuthentication();
builder.Services.AddAuthorization();
builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddFundraisingApplication();
builder.Services.AddFundraisingInfrastructure();
```

Finally:

EXAMPLE / FLOW
```
var app = builder.Build();
```

Conceptually:

EXAMPLE / FLOW
```
empty application builder
        |
        + authentication
        + authorization
        + DI
        + controllers
        + OpenAPI
        + database
        + logging
        |
        v
built application
```

• Test Builder Pattern

We should also use Builder for tests.

Example:

EXAMPLE / FLOW
```
var gift = new GiftBuilder()
    .ForTenant(1)
    .ForConstituent(1001)
    .WithAmount(1000m)
    .WithCampaign(200)
    .Build();
```

This is especially useful because domain entities can have many fields.

STRUCTURAL PATTERNS

4. Repository Pattern

You specifically want:

EXAMPLE / FLOW

```
GenericRepository<T>
IRepository<T>
```

We can include them, but carefully.

A generic repository can handle simple shared operations.

Example:

EXAMPLE / FLOW

```
public interface IRepository<TEntity>
    where TEntity : class
{
    Task<TEntity?> GetByIdAsync(
        long id,
        CancellationToken cancellationToken);

    void Add(TEntity entity);

    void Remove(TEntity entity);
}
```

Possible use:

EXAMPLE / FLOW

```
Campaign
Fund
Task
```

for simple CRUD.

Specialized Repository

Complex aggregates deserve specialized repositories.

Example:

EXAMPLE / FLOW

```
public interface IGiftRepository
{
    Task<Gift?> GetAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken);

    Task<IReadOnlyList<Gift>>
```

```
        GetConstituentGiftsAsync(
            long tenantId,
            long constituentId,
            CancellationToken cancellationToken);

    void Add(Gift gift);
}
```

Why?

Gift has:

EXAMPLE / FLOW

```
GiftAllocation

Campaign relationship

Constituent relationship

transaction rules
```

that a generic repository may not express well.

Recommended architecture:

EXAMPLE / FLOW

```
GenericRepository<T>
→ simple shared persistence

Specialized Repository
→ complex aggregate queries
```

• Repository Business Flow

EXAMPLE / FLOW

```
CreateGiftHandler
      |
      v
IGiftRepository
      |
      v
GiftRepository
      |
      v
FundraisingDbContext
      |
      v
FundraisingDb
```

Application does not know EF Core implementation details.

• 5. Unit of Work Pattern

A Unit of Work represents multiple data modifications that should be persisted together.

Example:

EXAMPLE / FLOW

```
Create Gift
   |
   +---- Gift
   +---- GiftAllocation
   +---- OutboxMessage
```

All should commit together.

Interface:

EXAMPLE / FLOW

```
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync(
        CancellationToken cancellationToken);
}
```

EF Core `DbContext` already provides much of Unit of Work behavior.

We can expose it through our interface for application-layer independence.

• Important Microservice Rule

Do not create:

EXAMPLE / FLOW

```
GlobalUnitOfWork
```

covering:

EXAMPLE / FLOW

```
FundraisingDb
PaymentDb
ReportingDb
```

Instead:

EXAMPLE / FLOW

```
FundraisingUnitOfWork
→ FundraisingDb

PaymentUnitOfWork
→ PaymentDb
```

Each microservice controls its own transaction.

• 6. Decorator Pattern

A decorator wraps a service and adds functionality without changing the original implementation.

Excellent example:

EXAMPLE / FLOW

```
IPaymentGateway
```

Actual implementation:

EXAMPLE / FLOW

```
ProviderAPaymentGateway
```

Decorators:

EXAMPLE / FLOW

```
LoggingPaymentGatewayDecorator

MetricsPaymentGatewayDecorator
```

Flow:

EXAMPLE / FLOW

```
CreatePaymentHandler
        |
        v
MetricsDecorator
        |
        v
LoggingDecorator
        |
        v
ProviderAPaymentGateway
        |
        v
Payment Provider
```

The core gateway does not become full of:

EXAMPLE / FLOW

```
logging

metrics

auditing
```

code.

• EmailServiceDecorator

A similar decorator pattern can be used later.

Example:

EXAMPLE / FLOW

```
IEmailService
      |
      v
EmailAuditDecorator
      |
      v
AzureCommunicationEmailService
```

The decorator:

EXAMPLE / FLOW

```
sends email
+
records audit event
```

without changing the actual email provider adapter.

---

7. Adapter Pattern

This is extremely important for external providers.

Our Application layer expects:

EXAMPLE / FLOW

```
IPaymentGateway
```

But Provider A SDK may expose:

EXAMPLE / FLOW

```
CreateCharge()

RefundCharge()

GetTransaction()
```

Provider B may expose completely different APIs.

Adapters convert those external APIs into our internal contract.

EXAMPLE / FLOW

```
Payments Application
        |
        v
IPaymentGateway
        |
        v
Provider A Adapter
        |
        v
Provider A SDK/API
```

and:

EXAMPLE / FLOW

```
Payments Application
        |
        v
IPaymentGateway
        |
        v
Provider B Adapter
        |
        v
Provider B SDK/API
```

Now external provider changes do not spread throughout the application.

• HttpClientAbstract

Your existing project uses something similar to:

EXAMPLE / FLOW

```
HttpClientAbstract
```

We can implement the same idea more specifically.

Instead of one enormous generic HTTP abstraction, prefer:

EXAMPLE / FLOW

```
IPaymentProviderClient

IIdentityProviderClient

IReportingExportClient
```

implemented with typed `HttpClient`.

That keeps contracts meaningful.

• 8. Facade Pattern

A Facade presents a simpler interface over multiple complicated operations.

Suppose Payment Processing requires:

EXAMPLE / FLOW

```
IdempotencyService

FraudService

PaymentGateway

PaymentRepository

Outbox

Audit
```

The API controller should not coordinate every component itself.

Instead:

EXAMPLE / FLOW

```
PaymentsController
       |
       v
IPaymentApplicationService
       |
       +---- Idempotency
       +---- Gateway
       +---- Repository
       +---- Fraud
       +---- Outbox
```

The application service becomes a facade over the payment use case.

Fundraising Facade Example

EXAMPLE / FLOW

```
IConstituentApplicationService
```

may provide:

EXAMPLE / FLOW

```
CreateConstituent

UpdateConstituent

GetSummary
```

while hiding:

EXAMPLE / FLOW

```
repositories

mapping

authorization helpers

queries
```

from the controller.

• Important Facade Rule

Do not create:

EXAMPLE / FLOW

```
IApplicationService
```

with 150 methods.

Use feature/domain-specific facades.

• 9. Proxy Pattern

A Proxy controls access to another object.

Your existing project uses lazy repository initialization inside UoW.

Conceptually:

EXAMPLE / FLOW

```
UnitOfWork.Gifts
      |
      v
Repository Proxy/Lazy
      |
      v
GiftRepository created only when needed
```

This works.

Example:

EXAMPLE / FLOW

```
public IGiftRepository Gifts =>
    _giftRepository ??=
        new GiftRepository(_dbContext);
```

This demonstrates a form of lazy proxy.

However, in our production-style design I would normally prefer constructor DI:

EXAMPLE / FLOW

```
GiftHandler
→ IGiftRepository
```

because it makes dependencies explicit.

Therefore:

EXAMPLE / FLOW

```
Lazy repository proxy
→ useful learning implementation

Explicit DI
→ preferred normal business code
```

Better Proxy Business Example

Caching proxy:

EXAMPLE / FLOW

```
ICampaignQueryService
        |
        v
CachedCampaignQueryProxy
        |
        v
CampaignQueryService
```

The proxy can:

EXAMPLE / FLOW

```
check cache

call real service if missing

return result
```

That is a strong practical Proxy use.

BEHAVIORAL PATTERNS

10. Strategy Pattern

Strategy represents interchangeable algorithms/behaviors.

Excellent payment example:

EXAMPLE / FLOW

```
IPaymentStrategy
       |
       +---- CardPaymentStrategy
       |
       +---- WalletPaymentStrategy
       |
       +---- DirectDebitStrategy
```

At runtime:

EXAMPLE / FLOW

```
PaymentMethodType
      |
      v
Strategy selected
      |
      v
appropriate payment workflow
```

Fraud Strategy

Another example:

EXAMPLE / FLOW

```
IFraudStrategy
     |
     +---- StandardFraudStrategy
     |
     +---- HighValueGiftFraudStrategy
```

Use this only if the business truly has multiple algorithms.

Exception Strategy

Another exception-handling strategy example:

EXAMPLE / FLOW

```
ICustomExceptionProvider
```

with multiple exception handlers can fit.

Example:

EXAMPLE / FLOW

```
Exception
    |
    +---- ValidationExceptionHandler
    |
    +---- NotFoundExceptionHandler
    |
    +---- ConcurrencyExceptionHandler
    |
    +---- PaymentProviderExceptionHandler
```

ASP.NET Core's `IExceptionHandler` architecture already naturally supports a strategy/chain style.

11. Template Method Pattern

Template Method defines the overall processing algorithm in a base class but allows subclasses to customize certain steps.

This is especially useful for background processing.

Example:

EXAMPLE / FLOW

```
public abstract class BatchProcessor<T>
{
    public async Task ProcessAsync(
        CancellationToken cancellationToken)
    {
        var items =
            await LoadBatchAsync(cancellationToken);

        foreach (var item in items)
        {
            await ProcessItemAsync(
                item,
                cancellationToken);
        }

        await CompleteBatchAsync(
            cancellationToken);
    }

    protected abstract Task<IReadOnlyList<T>>
        LoadBatchAsync(
            CancellationToken cancellationToken);

    protected abstract Task ProcessItemAsync(
        T item,
        CancellationToken cancellationToken);

    protected virtual Task CompleteBatchAsync(
        CancellationToken cancellationToken)
        => Task.CompletedTask;
}
```

Then:

EXAMPLE / FLOW

```
OutboxProcessor
WebhookProcessor
ReportingETLProcessor
```

can share the workflow.

• Azure Function Template Method

Your existing:

EXAMPLE / FLOW

```
BaseFunction
```

concept can also be used later in Azure.

Common algorithm:

EXAMPLE / FLOW

```
start trace

validate input

execute function

handle exception

record metric

finish trace
```

Specific Azure Functions implement only the operation itself.

Do not build a base class if functions share almost nothing.

• 12. Chain of Responsibility

ASP.NET Core middleware is one of the best Chain of Responsibility examples.

Flow:

EXAMPLE / FLOW

```
HTTP Request
      |
      v
Exception Middleware
      |
      v
Correlation Middleware
      |
      v
Authentication
      |
      v
Authorization
      |
      v
Rate Limiting
      |
      v
Controller
```

Each step decides:

EXAMPLE / FLOW

```
handle request
or
pass to next step
```

That is Chain of Responsibility.

• Custom Authorization Middleware

Your existing:

EXAMPLE / FLOW

```
AuthorizeMiddleware
```

can be studied.

However, normal ASP.NET Core authorization should usually use:

EXAMPLE / FLOW

```
Authentication middleware

Authorization middleware

Policies

Requirements

Handlers
```

rather than creating custom authorization logic for everything.

Custom middleware is appropriate when there is a genuinely cross-cutting requirement not already represented by standard authorization.

13. Observer Pattern

Observer means interested components react when something happens.

Inside one process:

EXAMPLE / FLOW

```
Gift Created
      |
      v
Domain Event
      |
      +---- Audit Handler
      |
      +---- Notification Handler
```

This is Observer-style behavior.

Distributed Observer

Across microservices:

EXAMPLE / FLOW

```
GiftCreated
      |
      v
Azure Service Bus
      |
      +---- Payment Service
      |
      +---- Reporting Consumer
      |
      +---- Notification Consumer
```

This is event-driven publish/subscribe architecture and conceptually related to Observer.

Email Audit Example

An event-driven email-auditing example:

EXAMPLE / FLOW

```
Event-driven email auditing
```

can become:

EXAMPLE / FLOW

```
ThankYouEmailSent
       |
       +---- Audit handler
       |
       +---- Engagement handler
```

One event can notify multiple interested consumers.

ARCHITECTURAL / APPLICATION PATTERNS

14. Dependency Injection

This is fundamental.

Instead of:

EXAMPLE / FLOW

```
var repository =
    new GiftRepository(
        new FundraisingDbContext(...));
```

we register:

EXAMPLE / FLOW

```
services.AddScoped<
    IGiftRepository,
    GiftRepository>();
```

Then:

EXAMPLE / FLOW

```
public CreateGiftHandler(
    IGiftRepository repository)
```

receives its dependency.

DI gives:

EXAMPLE / FLOW

```
loose coupling

testing

replaceability

lifetime management
```

15. Service Layer Pattern

Controllers should not contain business orchestration.

Use:

EXAMPLE / FLOW

```
Application Services
or
Command/Query Handlers
```

Example:

EXAMPLE / FLOW

```
GiftsController
      |
      v
CreateGiftService/Handler
      |
      v
business workflow
```

Service layer responsibilities:

EXAMPLE / FLOW

```
coordinate repositories

call domain behavior

manage use-case flow

call external abstractions
```

16. DTO Pattern

Never expose database entities directly.

Use:

EXAMPLE / FLOW

```
Request DTO

Response DTO
```

Example:

EXAMPLE / FLOW

```
public sealed record CreateGiftRequest(
    long ConstituentId,
    long? CampaignId,
    decimal TotalAmount,
    IReadOnlyList<GiftAllocationRequest> Allocations);
```

Response:

EXAMPLE / FLOW

```
public sealed record GiftResponse(
    long GiftId,
    decimal TotalAmount,
    string CurrencyCode);
```

Reasons:

EXAMPLE / FLOW

```
security

versioning

API stability

validation

separation from persistence model
```

17. Mapper Pattern

AutoMapper profiles are one option for this project.

We can use AutoMapper if you want the learning experience.

Example:

EXAMPLE / FLOW

```
CreateConstituentRequest
       |
       v
AutoMapper
       |
       v
CreateConstituentCommand
```

and:

EXAMPLE / FLOW

```
Constituent
     |
     v
Mapper
     |
     v
ConstituentResponse
```

Example profiles:

EXAMPLE / FLOW

```
ConstituentProfile

GiftProfile

CampaignProfile

PaymentProfile

SettlementProfile
```

Important Mapper Rule

Do not hide complicated business behavior inside AutoMapper mappings.

Bad:

EXAMPLE / FLOW

```
AutoMapper
calculates business rules
calls database
calls Payment API
```

Mapping should mostly transform shapes.

Complex business decisions belong in handlers/domain services.

18. Options Pattern

Use strongly typed configuration.

Example:

EXAMPLE / FLOW

```
public sealed class PaymentProviderOptions
{
    public required string BaseUrl { get; init; }

    public int TimeoutSeconds { get; init; }
}
```

Use:

EXAMPLE / FLOW

```
IOptions<T>

IOptionsSnapshot<T>

IOptionsMonitor<T>
```

depending on lifecycle.

IOptionsSnapshot

Good for scoped configuration scenarios.

Example:

EXAMPLE / FLOW

```
Tenant-configurable runtime settings
```

where configuration provider supports reload.

Do not automatically choose snapshot for every setting.

MVC Pattern

ASP.NET Core controller APIs use parts of MVC.

Traditional MVC:

EXAMPLE / FLOW

```
Model
Controller
View
```

Our Web API is primarily:

EXAMPLE / FLOW

```
Model/DTO
+
Controller
+
JSON response
```

There is no Razor/View UI because:

EXAMPLE / FLOW

```
Angular
```

is our frontend.

Therefore:

EXAMPLE / FLOW

```
ASP.NET Web API
uses MVC controller infrastructure

Angular
provides the actual UI
```

Do not create Razor Views simply to demonstrate MVC.

Policy Scheme Pattern

This is particularly useful in authentication systems.

Imagine API may accept:

EXAMPLE / FLOW

```
Employee Entra tokens

Partner tokens
```

A policy scheme can inspect a request and forward authentication to the appropriate handler.

Conceptually:

EXAMPLE / FLOW

```
Incoming Token
      |
      v
Authentication Policy Scheme
      |
      +---- Employee Entra Scheme
      |
      +---- Partner Scheme
```

For our first implementation:

EXAMPLE / FLOW

```
one Microsoft Entra API scheme
```

is enough.

Introduce policy schemes later only if the business genuinely supports multiple authentication schemes.

• Exception Handler Pattern

Use centralized:

EXAMPLE / FLOW

```
IExceptionHandler
+
ProblemDetails
```

Mapping:

EXAMPLE / FLOW

```
EntityNotFoundException
→ 404

BusinessRuleException
→ 400 / 409

ConcurrencyConflictException
→ 409

PaymentProviderException
→ suitable gateway/server response

unexpected exception
→ 500
```

This is preferable to repeating `try/catch` in every controller.

ADDITIONAL PATTERNS WE SHOULD USE

Your existing project patterns are useful, but our architecture needs several others.

19. CQRS Pattern

CQRS separates:

EXAMPLE / FLOW

```
Commands
```

from:

EXAMPLE / FLOW

```
Queries.
```

Command:

EXAMPLE / FLOW

```
CreateGiftCommand
```

changes data.

Query:

EXAMPLE / FLOW

```
GetGiftQuery
```

reads data.

Example:

EXAMPLE / FLOW

```
POST /gifts
      |
      v
CreateGiftCommand
      |
      v
CreateGiftHandler
```

while:

EXAMPLE / FLOW

```
GET /gifts/100
      |
      v
GetGiftQuery
      |
      v
GetGiftHandler
```

This is extremely useful because our write and reporting models have different requirements.

20. Command Pattern

CQRS commands are also examples of Command Pattern.

Object:

EXAMPLE / FLOW

```
CreateGiftCommand
```

represents the request:

EXAMPLE / FLOW

```
Create this Gift.
```

The handler executes it.

This provides:

EXAMPLE / FLOW

```
clear use cases

testing

logging

validation pipelines
```

21. Mediator Pattern

Instead of controller directly referencing:

EXAMPLE / FLOW

```
CreateGiftHandler

UpdateGiftHandler

DeleteGiftHandler

SearchGiftHandler
```

it can send requests through a mediator.

EXAMPLE / FLOW

```
Controller
    |
    v
Mediator
    |
    v
Correct Handler
```

This reduces controller coupling.

Do not introduce Mediator merely to add another layer.

It becomes valuable when the application has many command/query handlers.

22. Result Pattern

Expected business failures should not always become exceptions.

Example:

EXAMPLE / FLOW

```
Gift allocations total $900
but Gift amount is $1,000.
```

That is expected validation.

Return:

EXAMPLE / FLOW

```
Result<CreateGiftResult>
```

containing:

EXAMPLE / FLOW

```
Success
or
Business Error
```

Example:

EXAMPLE / FLOW

```
public sealed record Error(
    string Code,
    string Message);
```

Then:

EXAMPLE / FLOW

```
Result.Fail(
    "Gift.InvalidAllocationTotal")
```

can map predictably to HTTP response.

23. Specification Pattern

Suppose many places need:

EXAMPLE / FLOW

```
Open overdue Tasks
```

Rule:

EXAMPLE / FLOW

```
CompletedAtUtc IS NULL
AND
DueDateUtc < now
```

Instead of duplicating the rule in many places, create:

EXAMPLE / FLOW

```
OverdueTasksSpecification
```

Another:

EXAMPLE / FLOW

```
ActiveCampaignSpecification
```

Use this only if query rules are genuinely reused.

Don't wrap every `.Where()` in a specification.

24. Domain Event Pattern

Domain events represent meaningful events inside one bounded context.

Example:

EXAMPLE / FLOW

```
GiftRecordedDomainEvent
```

When Gift creation completes, internal handlers may:

EXAMPLE / FLOW

```
create audit entry

create thank-you Task

prepare external integration event
```

Domain event:

EXAMPLE / FLOW

```
inside service
```

Integration event:

EXAMPLE / FLOW

```
between services
```

Do not confuse them.

25. Outbox Pattern

This is essential.

Problem:

EXAMPLE / FLOW

```
Save Gift succeeds

Service Bus publish fails
```

Now database and messaging are inconsistent.

Solution:

EXAMPLE / FLOW

```
BEGIN DATABASE TRANSACTION

Insert Gift

Insert GiftAllocation

Insert OutboxMessage

COMMIT
```

Then worker publishes later.

Flow:

EXAMPLE / FLOW

```
Gift
+
Outbox
      |
      v
Commit
      |
      v
Outbox Worker
      |
      v
Service Bus
```

This is one of the most important patterns in your microservice architecture.

---

26. Inbox / Idempotent Consumer Pattern

Service Bus may deliver the same event more than once.

Payment service receives:

EXAMPLE / FLOW

```
GiftCreated
EventId = ABC
```

Before processing:

```
EXAMPLE / FLOW
Have we already processed ABC?
```

If yes:

```
EXAMPLE / FLOW
ignore duplicate.
```

If no:

```
EXAMPLE / FLOW
process
+
record ABC as processed.
```

This is often called:

```
EXAMPLE / FLOW
Inbox Pattern
```

or:

```
EXAMPLE / FLOW
Idempotent Consumer Pattern.
```

You should add an integration table such as:

```
EXAMPLE / FLOW
ProcessedMessage
```

to each service that consumes important events.

---

27. Idempotency Pattern

Especially important for:

```
EXAMPLE / FLOW
POST /payments
POST /refunds
webhooks
```

Example:

```
EXAMPLE / FLOW
Idempotency-Key:
ABC123
```

Two identical requests should not create two Payments.

Flow:

EXAMPLE / FLOW

```
request
   |
   v
check/insert unique key
   |
   +---- already processed
   |        |
   |        v
   |   return previous result
   |
   v
perform operation
```

SQL unique indexes provide final concurrency protection.

28. Saga / Process Manager Pattern

Once Fundraising and Payment are separate services, you cannot use one SQL transaction for everything.

Example business workflow:

EXAMPLE / FLOW

```
Gift Recorded
      |
      v
Payment Requested
      |
      v
Payment Processing
      |
  +---+---+
  |       |
success  failure
```

A process manager can track the overall workflow.

Example:

EXAMPLE / FLOW

```
Gift Created
      |
      v
GiftPaymentProcess
      |
      v
PaymentRequested
      |
      v
PaymentSucceeded
```

or:

EXAMPLE / FLOW

```
PaymentFailed
```

The workflow reacts accordingly.

Use Saga when the distributed business process becomes complex enough to need explicit orchestration/compensation.

Do not implement Saga on day one for simple event consumption.

29. Retry Pattern

External dependencies sometimes fail temporarily.

Example:

EXAMPLE / FLOW

```
Payment provider
→ HTTP 503
```

Retry may help.

But:

EXAMPLE / FLOW

```
retry payment charge
```

can be dangerous unless the provider supports idempotency.

Use retries only when the operation is safe/idempotent.

30. Circuit Breaker Pattern

Suppose Payment Provider is unavailable.

Without circuit breaker:

EXAMPLE / FLOW

```
Request 1 fails

Request 2 fails

Request 3 fails

Request 4 fails

10,000 requests keep hitting provider
```

Circuit breaker:

EXAMPLE / FLOW

```
Repeated failures
      |
      v
Circuit OPEN
      |
      v
fail fast
      |
      v
wait
      |
      v
test provider again
```

This protects our application and the failing provider.

31. Timeout Pattern

Every external call must have a maximum wait.

Example:

EXAMPLE / FLOW

```
Payment provider request
```

must not wait forever.

Use:

EXAMPLE / FLOW

```
Timeout
```

with clear cancellation.

32. Bulkhead Pattern

Bulkhead isolates resources.

Imagine:

EXAMPLE / FLOW

```
Reporting export
```

starts hundreds of expensive operations.

It should not consume every resource needed by:

EXAMPLE / FLOW

```
Create Gift
```

or:

EXAMPLE / FLOW

```
Process Payment.
```

You can isolate:

EXAMPLE / FLOW

```
thread/concurrency limits

connection resources

worker pools
```

so one workload does not bring down the entire platform.

This becomes more relevant at larger scale.

33. Cache-Aside Pattern

Instead of database automatically controlling application cache:

EXAMPLE / FLOW

```
Application
    |
    v
Check Redis
    |
 +--+--+
 |     |
hit   miss
 |     |
 v     v
return DB query
       |
       v
     cache
       |
       v
     return
```

Possible cached information:

EXAMPLE / FLOW

```
Campaign list

Fund metadata

short-lived reporting summary
```

Do not cache Payment state aggressively.

34. API Gateway Pattern

Later Azure architecture:

EXAMPLE / FLOW

```
Angular
   |
   v
API Management
   |
   +---- Fundraising API
   |
   +---- Payment API
   |
   +---- Reporting API
```

Angular does not need to know:

EXAMPLE / FLOW

```
three hostnames

service topology

internal authentication details
```

The gateway provides:

EXAMPLE / FLOW

```
single entry point

routing

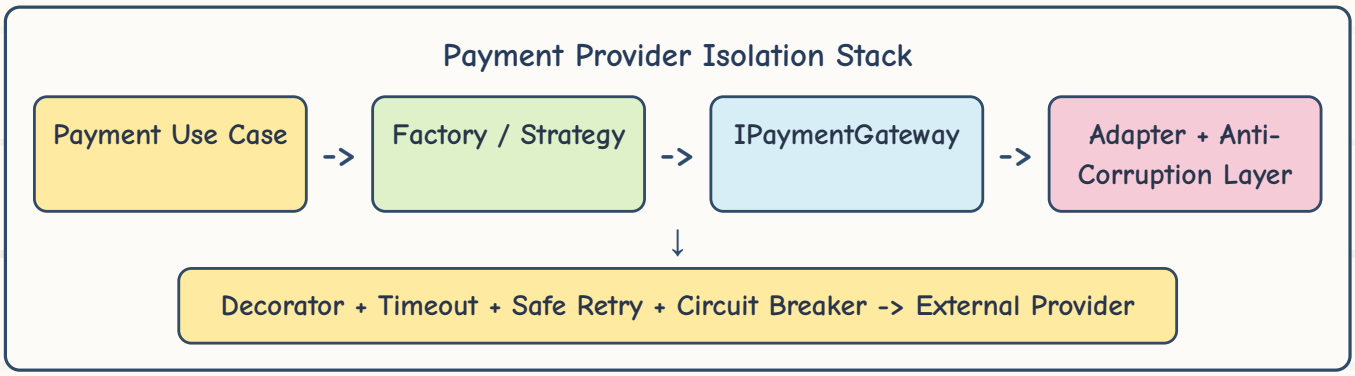
rate limiting

policies

API versioning exposure

monitoring
```

35. Anti-Corruption Layer



This is extremely useful around external payment systems.

Our domain speaks:

EXAMPLE / FLOW

```
Payment

PaymentAttempt

Refund

Settlement
```

Provider may call them:

EXAMPLE / FLOW

```
Charge

Intent

Capture

Payout
```

Do not allow provider vocabulary to invade the entire Payments domain.

Use:

EXAMPLE / FLOW

```
Payment Provider Adapter
+
Anti-Corruption Layer
```

to translate:

EXAMPLE / FLOW

```
Provider Charge
        |
        v
Our Payment
```

This makes changing providers much easier.

---

36. State Pattern — Optional Advanced Pattern

Payment has states such as:

EXAMPLE / FLOW

```
Pending

Processing

Succeeded

Failed

PartiallyRefunded

Refunded
```

You could implement state behavior using:

EXAMPLE / FLOW

```
PaymentState
```

classes.

Example:

EXAMPLE / FLOW

```
PaymentState
      |
      +---- PendingState
      +---- ProcessingState
      +---- SucceededState
      +---- RefundedState
```

Then:

EXAMPLE / FLOW

```
SucceededState
```

allows:

EXAMPLE / FLOW

```
Refund
```

while:

EXAMPLE / FLOW

```
FailedState
```

does not.

This is useful to learn.

However, for the initial project a simpler:

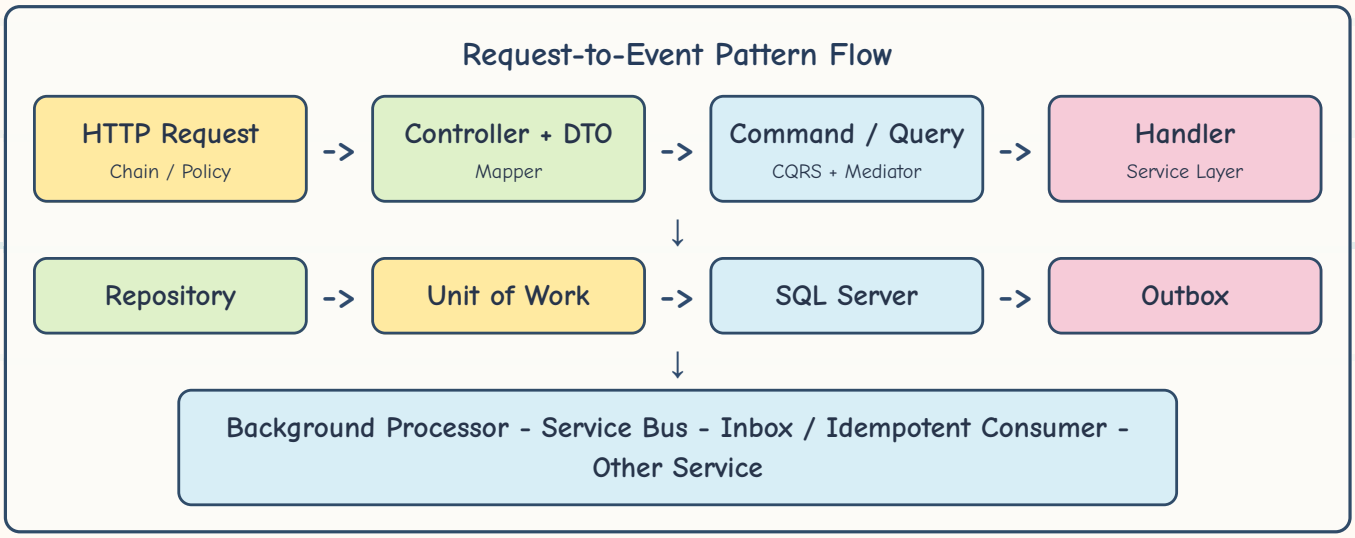
EXAMPLE / FLOW

```
enum/status
+
domain validation
```

may be easier.

Implement State Pattern later as an advanced refactoring exercise.

COMPLETE FUNDRAISING BUSINESS FLOW WITH PATTERNS



Now combine everything.

User sends:

EXAMPLE / FLOW

```
POST /api/v1/gifts
```

Request enters:

EXAMPLE / FLOW

```
Middleware Pipeline
```

Pattern:

EXAMPLE / FLOW

```
Chain of Responsibility
```

Then:

EXAMPLE / FLOW

```
Authentication
Authorization
```

Pattern:

EXAMPLE / FLOW

```
Policy
```

Controller receives:

EXAMPLE / FLOW

```
CreateGiftRequest
```

Pattern:

EXAMPLE / FLOW

```
DTO
```

Mapping:

EXAMPLE / FLOW

```
CreateGiftRequest
       |
       v
CreateGiftCommand
```

Pattern:

EXAMPLE / FLOW

```
Mapper
```

Controller sends command:

EXAMPLE / FLOW

```
CreateGiftCommand
```

Patterns:

EXAMPLE / FLOW

```
Command
CQRS
Mediator
```

Handler runs:

EXAMPLE / FLOW

```
CreateGiftHandler
```

Pattern:

EXAMPLE / FLOW

```
Service Layer
```

Dependencies provided through:

EXAMPLE / FLOW

```
DI
```

Persistence:

EXAMPLE / FLOW

```
IGiftRepository
```

Pattern:

EXAMPLE / FLOW

```
Repository
```

Commit:

EXAMPLE / FLOW

```
IUnitOfWork
```

Pattern:

EXAMPLE / FLOW

```
Unit of Work
```

During transaction:

EXAMPLE / FLOW

```
Gift

GiftAllocation

OutboxMessage
```

Pattern:

EXAMPLE / FLOW

```
Outbox
```

After commit:

EXAMPLE / FLOW

```
GiftRecordedDomainEvent
```

Pattern:

EXAMPLE / FLOW

```
Observer / Domain Event
```

Outbox worker:

EXAMPLE / FLOW

```
BackgroundService
```

possibly using:

EXAMPLE / FLOW

```
Template Method
```

creates context through:

EXAMPLE / FLOW

```
IDbContextFactory
```

Pattern:

EXAMPLE / FLOW

```
Factory
```

Publishes:

EXAMPLE / FLOW

```
GiftCreated
```

Payment Service consumes it.

Before processing:

EXAMPLE / FLOW

```
ProcessedMessage
```

Pattern:

EXAMPLE / FLOW

```
Inbox / Idempotent Consumer
```

Payment handler chooses:

EXAMPLE / FLOW

```
Provider A
or
Provider B
```

Patterns:

EXAMPLE / FLOW

```
Factory
Strategy
```

Provider integration:

EXAMPLE / FLOW

```
IPaymentGateway
```

Patterns:

EXAMPLE / FLOW

```
Adapter
Anti-Corruption Layer
```

Gateway wrapped by:

EXAMPLE / FLOW

```
Logging decorator
Metrics decorator
```

Pattern:

EXAMPLE / FLOW

```
Decorator
```

HTTP request uses:

EXAMPLE / FLOW

```
Timeout
Retry where safe
Circuit Breaker
```

Patterns:

EXAMPLE / FLOW

```
Resilience patterns
```

Payment succeeds.

Payment Service creates:

EXAMPLE / FLOW

```
PaymentSucceeded
```

through its own Outbox.

Reporting and Fundraising consumers observe it.

Pattern:

EXAMPLE / FLOW

```
Observer / Pub-Sub
```

That is how design patterns should work together in a real business system.

DESIGN PATTERNS BY MICROSERVICE

Fundraising Service

Primary patterns:

EXAMPLE / FLOW

```
Clean Architecture

DI

Repository

Unit of Work

CQRS

Command

Mediator

DTO

Mapper

Service Layer

Result

Exception Handler

Chain of Responsibility

Outbox

Domain Event

Observer

Specification

Builder

Options

Cache-Aside
```

Payment Service

This service has the richest design-pattern usage.

Use:

EXAMPLE / FLOW DI

Repository

Unit of Work

Factory

Strategy

Adapter

Anti-Corruption Layer

Decorator

Facade

CQRS

Command

Mediator

DTO

Mapper

Result

Exception Handler

Outbox

Inbox

Idempotency

Retry

Timeout

Circuit Breaker

Saga later

State Pattern later

Options

Builder

Reporting Service

Reporting should remain simpler.

Use:

EXAMPLE / FLOW

```
CQRS queries

DTO

Mapper

Facade

Repository/query abstraction where useful

Dapper

Cache-Aside

Options

DI

Decorator for metrics

Builder

Specification where reusable

Proxy/caching proxy where useful
```

Do not force:

EXAMPLE / FLOW

```
Unit of Work
```

onto read-only reports without a reason.

• Patterns Used Mainly by Infrastructure

EXAMPLE / FLOW

```
Factory
Adapter
Decorator
Proxy
Options
Retry
Circuit Breaker
Cache-Aside
```

Patterns Used Mainly by Application Layer

EXAMPLE / FLOW

Service Layer
CQRS
Command
Mediator
Result
Facade
Specification

Patterns Used Mainly by Domain

EXAMPLE / FLOW

Encapsulation

Strategy where domain-specific

Domain Event

State where needed

Factory where entity creation is complicated

Patterns Used Mainly by API

EXAMPLE / FLOW

MVC / Controller

DTO

Mapper

Chain of Responsibility

Exception Handler

Policy Authorization

API Versioning

DI

Patterns Used Between Microservices

EXAMPLE / FLOW

Outbox

Inbox

Idempotent Consumer

Pub/Sub / Observer

Saga / Process Manager

API Gateway

Retry

Circuit Breaker

Anti-Corruption Layer

Patterns We Should NOT Force Everywhere

Some patterns are valuable but become harmful when overused.

Generic Repository Everywhere

Do not hide every EF feature behind:

EXAMPLE / FLOW

```
GenericRepository<T>
```

Use specialized repositories for complex aggregates.

Factory Everywhere

Do not create factories for objects whose construction is trivial.

Singleton Everywhere

Most business services should not be singleton.

Default request/business lifetime should normally remain:

EXAMPLE / FLOW

```
Scoped
```

where appropriate.

Inheritance Everywhere

Prefer:

EXAMPLE / FLOW

```
composition
```

unless a real `is-a` relationship exists.

Strategy for One Algorithm

If there is only:

EXAMPLE / FLOW

```
one payment implementation
```

and no expected variation, an elaborate Strategy hierarchy may initially be unnecessary.

We will use Strategy where the project genuinely has variants.

Proxy Just for Lazy Repository Creation

You can implement it as a learning example because your existing project uses it.

But do not make hidden/lazy dependencies your default architecture.

Explicit constructor dependencies are usually easier to understand.

Template Method Everywhere

Only use when multiple processors genuinely share the same algorithm skeleton.

AutoMapper Everywhere

Simple mapping:

EXAMPLE / FLOW

```
new GiftResponse(
    gift.Id,
    gift.TotalAmount);
```

may be clearer than AutoMapper.

Use AutoMapper for repetitive mapping where it genuinely reduces noise.

Saga Too Early

Start with:

EXAMPLE / FLOW

```
events
+
Outbox
+
idempotent consumers
```

When the distributed workflow becomes genuinely complicated, introduce Saga/Process Manager.

FINAL RECOMMENDED PATTERN SET

For your final portfolio, I recommend demonstrating these as **real production patterns**:

EXAMPLE / FLOW Factory

Builder

Repository

Unit of Work

Decorator

Adapter

Facade

Strategy

Chain of Responsibility

Observer / Domain Events

Dependency Injection

Service Layer

DTO

Mapper

Options

MVC / Controller

Policy Authorization

Exception Handler

CQRS

Command

Mediator

Result

Outbox

Inbox

Idempotency

Retry

Timeout

```
Circuit Breaker

Cache-Aside

API Gateway

Anti-Corruption Layer
```

And demonstrate these as **advanced/selected patterns** where appropriate:

EXAMPLE / FLOW

```
Proxy

Template Method

Specification

Saga / Process Manager

Bulkhead

State Pattern
```

That gives you far more useful design-pattern experience than simply implementing all 23 Gang of Four patterns with artificial examples.

Final Architecture Memory Map

EXAMPLE / FLOW

```
HTTP REQUEST
    |
    v
Chain of Responsibility
Middleware
    |
    v
Authentication / Policy
    |
    v
Controller
MVC
    |
    v
DTO
    |
    v
Mapper
    |
    v
Command / Query
CQRS
    |
    v
Mediator
    |
    v
Service Layer / Handler
    |
    +----------------------+
    |                      |
    v                      v
Strategy                Facade
    |                      |
    v                      v
Factory                Domain
    |
    v
Adapter
    |
    v
Repository
    |
    v
Unit of Work
    |
    v
SQL Server
    |
    v
Outbox
    |
    v
Background Processor
Template Method
    |
```

```
    v
Service Bus
Observer / Pub-Sub
    |
    v
Other Microservice
    |
    v
Inbox / Idempotency
    |
    v
Business Handler
```

External integrations additionally use:

EXAMPLE / FLOW

```
Decorator

Adapter

Anti-Corruption Layer

Timeout

Retry

Circuit Breaker
```

Caching uses:

EXAMPLE / FLOW

```
Proxy
+
Cache-Aside
```

Distributed workflows can later use:

EXAMPLE / FLOW

```
Saga / Process Manager
```

Payment lifecycle can later demonstrate:

EXAMPLE / FLOW

```
State Pattern.
```

The main rule for the entire project is:

Each pattern should be attached to a real business or technical requirement. In our project, Payment Service naturally demonstrates Factory, Strategy, Adapter, Decorator, Anti-Corruption Layer, Idempotency and resilience patterns; Fundraising naturally demonstrates Repository, Unit of Work, CQRS, Domain Events and Outbox; Reporting demonstrates Query/CQRS, caching and optimized read models; while DI, DTO, Mapper, Options, Exception Handler and middleware/Chain of Responsibility are shared across all APIs.

That gives you a project where, in an interview, you can explain not only:

> "I know the Factory Pattern."

but:

> "In the Payment microservice, I used a Factory with Strategy and Adapter implementations to select the configured payment provider at runtime. The application layer depends on `IPaymentGateway`, while provider-specific SDK models are isolated behind adapters and an anti-corruption layer. Logging and metrics are added with decorators, and provider calls are protected with timeout/circuit-breaker policies. This means adding another provider doesn't require rewriting the payment business workflow."

That is the level of design-pattern understanding this project should give you.